

RELAZIONE PROGETTO HOMEWORK 1

Distributed Systems and Big Data

Anno Accademico: 2025-2026,

Studente: [ANTONIO NICOLO, SCARVAGLIERI, 1000016405]

1. Introduzione, Architettura e Obiettivi

L'obiettivo principale del progetto è la realizzazione e il testing di **un sistema a microservizi distribuito e dockerizzato**. Il sistema è composto da due servizi principali che operano su una rete Docker dedicata.

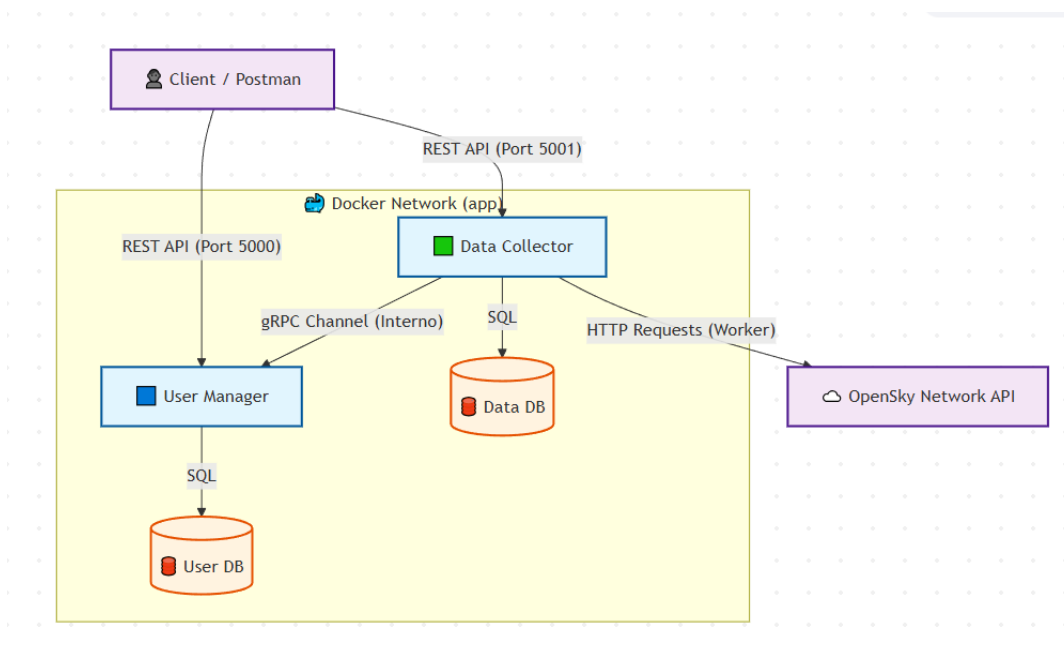
1.1 Obiettivi Funzionali

Il sistema deve essere in grado di:

- Gestire la registrazione e la cancellazione di utenti.
- Recuperare e l'elaborazione di dati su voli aerei in partenza e arrivo da OpenSky Network.

1.2 Struttura del Sistema

- **User Manager:** Un microservizio responsabile della gestione anagrafica degli utenti. Utilizza API **REST** per la registrazione (con politica *at-most-once*) e funge da server **gRPC** per fornire servizi di verifica interna. Persiste i dati su un database **PostgreSQL** dedicato.
- **Data Collector:** Un microservizio che gestisce le richieste di monitoraggio degli aeroporti. Agisce da client **gRPC** per validare l'esistenza degli utenti prima di attivare i servizi. Integra un **Worker in background** (Scheduler) che interroga ciclicamente l'API esterna di **OpenSky Network** per recuperare dati di volo reali (Arrivi/Partenze) e salva i dati in formato strutturato (**JSONB**) su un secondo database **PostgreSQL** dedicato.
- **Database (PostgreSQL):** Scelto per la sua affidabilità e per la capacità di gestire sia dati relazionali (per gli utenti) sia dati non relazionali attraverso il tipo **JSONB** garantendo una buona flessibilità.



2. Scelte di Design e Stack Tecnologico

2.1 Architettura e Containerizzazione

- **Docker e Docker Compose:** L'intero sistema è sviluppato come un'architettura dockerizzata per garantire l'isolamento dei servizi (Microservizi) e la portabilità dell'ambiente. Docker Compose è utilizzato per l'orchestrazione, la gestione delle reti interne e l'avvio simultaneo di tutti i componenti.
- **Linguaggio (Python/Flask):** Python è stato scelto per la rapidità di sviluppo e per l'ampia disponibilità di librerie robuste per gRPC. Flask funge da framework leggero per gli endpoint REST.

2.2 Pattern di Comunicazione Eterogenea

- **gRPC (Protocol Buffers):** Utilizzato per la comunicazione sincrona **interna** tra Data Collector e User Manager.
- **REST (HTTP/JSON):** Utilizzato per le interazioni esterne. Questo include sia la comunicazione **Client-Server** (es. Postman per la registrazione utenti) che la comunicazione **Service-to-External-API** (Data Collector per recuperare dati da OpenSky Network).

2.3 Gestione Dati

- **Politica "At-Most-Once":** Implementata sulla registrazione utenti. Il codice assicura l'idempotenza utilizzando la clausola SQL **ON CONFLICT DO NOTHING** sul campo email (chiave primaria) per prevenire l'inserimento di duplicati.

3. Directory

Di seguito la struttura utilizzata per il progetto :

```
progetto_dsbd/
|
├── docker-compose.yaml          # File di orchestrazione (User Manager, Data
Collector, DBs)
├── .gitignore                  # File esclusi da Git (es. .venv)
├── README.md                   # Documentazione di avvio
|
├── protos/                     # Cartella condivisa (Contratto gRPC)
|   └── usermanager.proto       # Definizione del servizio CheckUser
|
```

```

└─ user_manager/                                # Microservizio: Gestione Utenti
|
| └─ Dockerfile                                # Configurazione container Python
| └─ requirements.txt                          # Dipendenze (Flask, Grpc ecc)
|
| └─ usermanager_pb2.py                        # Codice generato gRPC
| └─ usermanager_pb2_grpc.py                  # Codice generato gRPC
|
| └─ app/                                       # Codice Sorgente Modulare
|   └─ __init__.py
|   └─ main.py                                # Entry point e avvio Flask/gRPC
|   └─ database.py                            # Connessione DB e creazione tabelle
|   └─ routes.py                              # Endpoint REST (/register, /delete)
|   └─ grpc_server.py                         # Implementazione logica Server gRPC
|
└─ data_collector/                             # Microservizio: Monitoraggio Voli
    |
    | └─ Dockerfile                            # Configurazione container Python
    | └─ requirements.txt                      # Dipendenze (Flask, gRPC, requests)
    |
    | └─ usermanager_pb2.py                    # Codice generato gRPC
    | └─ usermanager_pb2_grpc.py              # Codice generato gRPC
    |
    | └─ app/                                  # Codice Sorgente Modulare
    |   └─ __init__.py
    |   └─ main.py                            # Entry point e API REST (/monitor, /flights)
    |   └─ database.py                        # Gestione DB (Salvataggio JSONB)
    |   └─ grpc_client.py                     # Client gRPC per verifica utente
    |   └─ scheduler.py                       # Worker ciclico per download dati OpenSky

```